C++中字符串与字符数组的完美转换

在C++开发中，我们经常需要在 `std::string` **（C++字符串类）** 和 `char` **数组/C风格字符串**之间转换（比如调用C语言接口、旧代码兼容）。

我用**最简单的场景+代码**，把两种转换讲透，结合你之前学的 `\0`、`strcpy` 知识点，无缝衔接！

核心前提

1. **char数组 → string：超级简单**，自动转换，无需手动处理 `\0`
2. **string → char数组**：分**只读**和**可修改**两种场景，是重点
3. 必备头文件：

Code block

```
#include <iostream>
#include <string>   // string类
#include <cstring>  // strcpy 等C风格函数
using namespace std;
```

一、char数组 / const char* → string

这是**自动转换**，`string` 类的构造函数直接支持，会自动读取到 `\0` 为止。

3种常用写法

Code block

```
// 1. 定义char数组
char cArr[] = "Hello C++";  // 自带\0

// 方式1：直接初始化（最常用）
string str1 = cArr;

// 方式2：构造函数初始化
string str2(cArr);

```

```
// 方式3：赋值
string str3;
str3 = cArr;

// 输出测试
cout << str1 << endl;  // Hello C++
```

✅ **优势**：完全不用管内存、`\0`，编译器自动处理。

二、string → char数组（重点）

分 **2种核心场景**，一定要区分！

场景1：只读使用 → `c_str()` 函数（90%的场景）

`string` 提供 **`c_str()`** 方法，返回一个 **`const char*`** 类型的指针（指向字符串首地址，自带 `\0`）。

✅ 用途：打印、传参给C语言函数、只读访问。

❌ 限制：**不能修改内容**（因为是 `const`）。

示例

Code block

```
string str = "Hello World";

// 转换为 const char*
const char* cStr = str.c_str();

cout << cStr << endl;  // 直接输出：Hello World
```

场景2：需要修改内容 → 拷贝到 char数组

如果需要**修改字符串内容**，不能直接用 `c_str()`，必须用 `strcpy` 把内容**拷贝**到自定义的 `char` 数组中。

步骤

1. 创建足够大的 `char` 数组

2. 用 `strcpy` 拷贝（包含 `\0`）
3. 可自由修改数组内容

示例

Code block

```
string str = "Hello";

// 1. 创建char数组（大小：str长度+1，存\0）
char cArr[20];

// 2. 拷贝 string -> char数组
strcpy(cArr, str.c_str());

// 3. 可以自由修改！
cArr[0] = 'h';
cout << cArr << endl;  // 输出 hello
```

补充：`data()` 函数（C++11）

`str.data()` 和 `c_str()` 功能完全一致，也返回 `const char*`，现代C++通用：

Code block

```
const char* cStr = str.data();
```

三、完整可运行代码（整合所有转换）

直接复制运行，覆盖所有常用场景：

Code block

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

int main() {
    // ========== 1. char数组 → string ==========
    char charArr[] = "I love C++";
```

```
    string str1 = charArr;  // 自动转换
    cout << "char数组转string: " << str1 << endl;

    // ========== 2. string → const char* (只读) ==========
    string str2 = "Hello String";
    const char* cStr = str2.c_str();  // 核心方法
    cout << "string转const char*: " << cStr << endl;

    // ========== 3. string → 可修改的char数组 ==========
    string str3 = "Apple";
    char newCharArr[20];
    strcpy(newCharArr, str3.c_str());  // 拷贝
    newCharArr[0] = 'a';  // 修改内容
    cout << "string转char数组(修改后): " << newCharArr << endl;

    return 0;
}
```

运行结果

Code block

```
char数组转string: I love C++
string转const char*: Hello String
string转char数组(修改后): apple
```

四、新手必避3个坑（最重要！）

1. **`c_str()` 返回的是 `const` 指针，绝对不能修改**

Code block

```
string str = "abc";
// str.c_str()[0] = 'x'; ❌ 报错! 只读
```

2. **不要保存 `c_str()` 的指针**

`string` 对象修改/销毁后，`c_str()` 指针会失效，变成野指针。

3. **char数组空间一定要足够**

用 `strcpy` 时，数组大小 ≥ `string长度 + 1`（给 `\0` 留位置）。

五、极简总结（背会这几句就够）

1. **char数组 → string**：直接赋值/初始化（自动转）
2. **string → 只读char指针**：`str.c_str()`（最常用）
3. **string → 可修改char数组**：`strcpy(目标数组, str.c_str())`
4. 所有转换**自动处理** `\0`，无需手动添加

（注：文档部分内容可能由 AI 生成）